

Universidad ORT Uruguay

Facultad de Ingeniería

Evidencia del diseño y especificación de la API (interfaz de programación de aplicaciones)

Obligatorio 1 – Diseño de Aplicaciones 2: FarmeORT

Juan Ferreiro – 328344

Sebastián Luciardi – 274754

Andrés Bustamante – 302420

Grupo M6C – Tutores: Alexander Wieler y Franco Bonavita

2026

Repositorio: https://github.com/IngSoft-DA2/M6CD-ID-274754-328344-302420

Declaración de autoría

Nosotros, Juan Ferreiro, Sebastián Luciardi y Andrés Bustamante, declaramos que el trabajo que se presenta en esa obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el Obligatorio 1 de Diseño de Aplicaciones 2;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra;
- En la obra, hemos acusado recibo de las ayudas recibidas;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.

Juan Ferreiro, Sebastián Luciardi y Andrés Bustamante – 08/10/2026

Abstract

Este documento especifica la API REST de FarmeORT y justifica su diseño. Discute cómo la API cumple las restricciones de REST y las prácticas del curso, describe el mecanismo de autenticación con token Bearer opaco, justifica el uso de cada familia de códigos de estado y describe los recursos expuestos. El centro del documento es la tabla de las 38 operaciones: para cada una indica verbo, URI, recurso, acceso, headers, parámetros de query, body, respuesta y todos los códigos de estado con su código de error. Se completa con los JSON de cada request y response, el formato de los errores con el catálogo de los 61 códigos y el manejo de conflictos por concurrencia.

Palabras clave

API REST; especificación de la API; FarmeORT; autenticación con token Bearer; códigos de estado HTTP; ASP.NET Web API; Diseño de Aplicaciones 2

Índice

1. Alcance y recursos 4
1.1 Referencia contractual y URL base 4
1.2 Recursos expuestos 4
2. Autenticación, mensajes y estados HTTP 5
2.1 Autenticación y autorización: cómo funcionan 5
2.2 Headers, serialización y parámetros 7
2.3 Familias y errores transversales 7
2.4 Headers Location de creación 8
3. Criterios REST: por qué la API los cumple 9
3.1 Las restricciones de REST 9
3.2 Prácticas de diseño del curso 10
4. Tabla central de las operaciones vigentes 12
5. JSON compartidos: requests y responses 29
5.1 Requests R01–R11 29
5.2 Responses S01–S14 30
6. Formato de errores y conflictos 35
6.1 ErrorResponse y códigos de validación 35
6.2 Concurrencia y reconsulta 37
7. Uso de IA 38
8. Bibliografía 39

1. Alcance y recursos

Convención: los campos marcados con [REQ] son requeridos; los que no lo llevan son opcionales.

1.1 Referencia contractual y URL base

El contrato corresponde a la rama develop del repositorio [2] en el commit 34f3a94, que integra todas las funcionalidades de la entrega, y se verificó contra el código: la tabla tiene las 38 operaciones que publican los controllers, y el catálogo de errores, los 61 códigos que mapea el filtro de excepciones. Las decisiones que originan cada regla están en la bitácora de decisiones [4], y los casos que las prueban, en la colección de Postman [5].

URL base convencional: https://{host}/api/v1, con host y puerto del despliegue. El routing fija /api/v1. Las URI de la tabla se concatenan al origen sin duplicar el prefijo. La entrega no incluye un despliegue público: la API se ejecuta localmente.

Desarrollo (launchSettings.json): http://localhost:5294 con el perfil http o https://localhost:7070 con el perfil https. La colección de Postman de la carpeta Datos usa http://localhost:5294 como baseUrl.

1.2 Recursos expuestos

Tabla 1. Recursos conceptuales y propósito de sus representaciones.

Recurso	Descripción
Sesión	Credencial opaca de acceso, creada por login y cerrada por su propietario.
Jugador / Usuario	Registro público de Jugador y gestión administrativa de usuarios activos y sus roles.
Repertorio	Skills declaradas por el Jugador autenticado, con nivel y fecha de declaración.
Skill	Habilidad del catálogo con categoría, dificultad y estado activa.
Torneo	Competición con organizador, fechas, cupo y estados Abierto, EnCurso, Finalizado o Cancelado.
Skill obligatoria	Asociación entre torneo y skill; como máximo dos, condiciona inscripciones y lanzamientos.
Ronda	Agrupación numerada de batallas de un torneo. Se expone como recurso derivado, sin exigir una tabla propia.
Inscripción	Participación de un Jugador en un torneo: Aceptada, Cancelada o Eliminada.
Batalla	Enfrentamiento de dos jugadores: Pendiente, EnArbitraje, Finalizada o Cancelada.
Lanzamiento	Turno de uso de una skill durante una batalla; se registran hasta ocho, cuatro por jugador.
Fallo	Decisión del Organizador: ganador, comentario y variación ELO; elimina al perdedor de la competición.
Ranking	Proyección ordenada del ELO y las batallas ganadas/perdidas globales.
Cancelación / Finalización	Procesos de cambio de estado expuestos mediante POST a subrecursos del torneo o la inscripción.

2. Autenticación, mensajes y estados HTTP

2.1 Autenticación y autorización: cómo funcionan

La API separa dos preguntas. La autenticación responde quién hace el pedido; la autorización, si esa persona puede hacer esa operación sobre ese recurso. La primera se resuelve con un token de sesión opaco que el cliente manda en cada pedido; la segunda, en dos niveles: el rol, en un filtro de la WebAPI, y la relación con el recurso concreto, en el servicio. A continuación se describe el recorrido completo de un pedido.

1. La contraseña nunca se guarda. Al registrar un jugador (`POST /jugadores`) o dar de alta un usuario (`POST /usuarios`), el servicio genera una sal aleatoria de 16 bytes y deriva la contraseña con PBKDF2-SHA256 y 100.000 iteraciones. En la base queda solo `sal.hash`, en Base64. La sal hace que dos usuarios con la misma contraseña tengan hashes distintos, y las iteraciones encarecen probar contraseñas por fuerza bruta si alguien obtuviera la tabla.

2. Iniciar sesión crea un recurso Sesión. El cliente envía sus credenciales:

```
POST /api/v1/sesiones
Content-Type: application/json

{ "email": "ana@ejemplo.com", "contrasena": "ClaveSegura123!" }
```

El servicio busca al usuario por email, deriva la contraseña recibida con la sal guardada y compara el resultado con el hash en tiempo constante, para que el tiempo de respuesta no revele cuántos bytes coincidieron. Si el email no existe, el usuario está dado de baja o la contraseña no coincide, responde siempre lo mismo: 401 `INVALID_CREDENTIALS`, «Email o contraseña incorrectos», sin decir cuál de los datos falló. Si las credenciales son correctas, genera un token (un GUID aleatorio de 122 bits), guarda la fila (token, usuario) en la tabla Sesiones, cuya clave primaria es el token, y responde:

```
HTTP/1.1 201 Created
Location: /api/v1/sesiones/3f2b8c1e-7d4a-4b9e-9c2f-1a2b3c4d5e6f

{ "token": "3f2b8c1e-7d4a-4b9e-9c2f-1a2b3c4d5e6f", "usuarioId": 7, "rol": "Jugador" }
```

El token es opaco: no contiene datos del usuario ni una firma. Por sí solo no significa nada; lo que le da valor es la fila de la tabla Sesiones que lo asocia a un usuario.

3. Cada pedido protegido trae el token. El cliente lo envía en el header estándar de HTTP:

```
POST /api/v1/inscripciones
Authorization: Bearer 3f2b8c1e-7d4a-4b9e-9c2f-1a2b3c4d5e6f
Content-Type: application/json

{ "torneoId": 4 }
```

El filtro `AutenticacionAttribute`, un filtro de autorización de ASP.NET, se ejecuta antes que el controller y antes de leer el body, y hace estas comprobaciones en orden:

- Que el header exista y empiece con `Bearer `. Si no, responde 401 `INVALID_TOKEN`.
- Que lo que sigue sea un GUID válido. Si no, también 401, sin consultar la base: un texto que no puede ser un token no merece una consulta.

- Que exista una sesión con ese token y que su usuario no esté dado de baja. Si no, 401 `INVALID_TOKEN`.

Si todo es correcto, guarda en el contexto del pedido una `SesionAutenticada` con el token, el id del usuario y su rol. El controller toma la identidad de ahí y de ningún otro lado: en el ejemplo, el body solo dice a qué torneo inscribirse y el jugador es el dueño del token. Si el cliente agrega un `jugadorId` en el body, se ignora; la colección de Postman lo comprueba. Así nadie puede actuar en nombre de otro cambiando un número en el pedido.

4. Primer nivel de autorización: el rol. Cada acción declara qué roles la pueden usar con el atributo `[Autorizacion(...)]`, que hereda de `AutenticacionAttribute`: primero autentica y después compara el rol de la sesión con la lista permitida. Si el rol no está, responde 403 `FORBIDDEN`. Por ejemplo, `UsuariosController` tiene `[Autorizacion(Roles.Administrador)]`; `POST /inscripciones`, `[Autorizacion(Roles.Jugador)]`, y `POST /batallas/{id}/fallos`, `[Autorizacion(Roles.Organizador)]`. Los roles son constantes (`Roles.Jugador`, `Roles.Organizador`, `Roles.Administrador`) y no textos sueltos, así que un error de tipeo no compila.

5. Segundo nivel: la relación con el recurso. Tener el rol correcto no alcanza. Un Organizador solo falla las batallas de los torneos que tiene asignados, y un Jugador solo cancela sus propias inscripciones. Esa comprobación necesita datos del recurso (quién organiza este torneo, de quién es esta inscripción), así que la hace el servicio. Por ejemplo, `PermisosDelTorneo` responde 403 `NOT_TOURNAMENT_ORGANIZER` si el organizador no es el asignado; cancelar una inscripción ajena da 403 `NOT_REGISTRATION_OWNER`, y cerrar la sesión de otro usuario, 403 `NOT_SESSION_OWNER`. Separarlo así le da a cada parte una sola responsabilidad: el filtro sabe de roles y el servicio sabe de reglas de negocio.

6. Autenticación opcional en los recursos públicos. El listado y el detalle de torneos, sus rondas, sus skills obligatorias y el ranking se pueden consultar sin token, con `[AutenticacionOpcional]`. Sin el header, el pedido es anónimo y la API oculta los torneos Cancelados (404 `TOURNAMENT_NOT_FOUND` o fuera del listado). Si el header viene, tiene que ser válido: un token cerrado o inventado da 401 y no se trata como anónimo, porque eso escondería un error del cliente. Un usuario autenticado de cualquier rol sí ve los torneos Cancelados. En las batallas, el Jugador necesita una inscripción en el torneo, en cualquier estado; el Organizador, ser el asignado, y el Administrador las ve todas.

7. Cerrar sesión revoca el token en el momento. `DELETE /api/v1/sesiones/{token}` borra la fila de la sesión y responde 204; el siguiente pedido con ese token da 401. Un usuario solo cierra sus propias sesiones: una ajena da 403 `NOT_SESSION_OWNER` y una que no existe, 404 `SESSION_NOT_FOUND`. Cuando un administrador da de baja a un usuario, sus tokens dejan de servir, porque la validación del paso 3 rechaza las sesiones de usuarios dados de baja.

Por qué un token opaco y no JWT. Con un JWT, la identidad viaja firmada dentro del token y el servidor no consulta la base en cada pedido, pero un token emitido sigue siendo válido hasta que vence: cerrar sesión o dar de baja a un usuario no lo invalida sin agregar una lista de revocados. Con el token opaco, cerrar sesión o dar de baja tiene efecto inmediato, no hay una clave secreta que custodiar y el token no expone datos. El costo es una consulta por pedido, que busca por la clave primaria de la tabla Sesiones. La API no publica vencimiento ni renovación del token: la sesión dura hasta que el usuario la cierra o lo dan de baja.

2.2 Headers, serialización y parámetros

Content-Type describe el cuerpo enviado; usar application/json en las operaciones con Body. MVC también admite text/json y application/*+json. Un tipo no admitido, por ejemplo text/plain, genera 415 UNSUPPORTED_MEDIA_TYPE. Body omitido, JSON mal formado o validación fallida generan 400 VALIDATION_ERROR. Accept expresa el formato solicitado para la respuesta; application/json es la recomendación, no un requisito obligatorio. La configuración actual no rechaza Accept: application/xml con 406.

Las representaciones se serializan como JSON camelCase. Las respuestas con contenido declaran application/json, normalmente con charset=utf-8; 204 no lleva cuerpo. Las colecciones son arrays directos, sin wrapper. null se serializa y los arrays vacíos se representan con []. No se exige Content-Type cuando la operación no consume Body. No se documenta WWW-Authenticate: no es emitido por estos filtros.

Parámetros de ruta: {id}, {skillId}, {torneoId} y {numero} tienen restricción int32; {token}, GUID. No son query params. Un segmento incompatible con la restricción retorna 404 NOT_FOUND antes de ejecutar la operación. Los ids enteros 0/negativos pueden coincidir con la ruta y luego resultar inexistentes. El rango positivo solo se exige donde se indica expresamente.

Los filtros son opcionales salvo torneoId en GET /batallas. Texto vacío se ignora; nombre/apellido/nickname usan coincidencia parcial sin distinguir mayúsculas. En filtros de enum, los valores desconocidos o con distinta capitalización se ignoran. dificultad fuera de 1..5 también se ignora. Un tipo no convertible (limit=abc, activa=abc, fecha inválida) genera 400 VALIDATION_ERROR. Query params desconocidos se ignoran; no existen sort, OrderBy, direction ni fields.

Fechas de query: usar YYYY-MM-DD; el binder acepta fechas convertibles a .NET. Los límites son inclusivos (fechaInicio del torneo o día de inscripción). fechaDesde posterior a fechaHasta devuelve []; fechaHasta=9999-12-31 se admite en inscripciones. Las fechas de torneo no cambian el estado automáticamente.

2.3 Familias y errores transversales

Tabla 2. Familias HTTP y uso efectivo en FarmeORT.

Familia	Significado y aplicación
1xx	Información del protocolo. FarmeORT no define respuestas funcionales 1xx.
2xx	Éxito: 200 con contenido; 201 crea recurso/proceso y agrega Location; 204 confirma sin Body.
3xx	Redirección o validación de caché. No hay respuestas funcionales 3xx configuradas en esta API.
4xx	Problema de formato, autenticación, permiso, visibilidad o conflicto: 400, 401, 403, 404, 405, 409 y 415.
5xx	Fallo inesperado del servidor: 500 INTERNAL_ERROR. No expresa validación o reglas conocidas del dominio.

400 distingue errores de ModelState (VALIDATION_ERROR) de datos inválidos del dominio (INVALID_DATA). 401 significa ausencia/invalidez de credencial; en login, credenciales incorrectas. 403 supone identidad válida sin permiso. 404 identifica inexistencia o invisibilidad acordada. 409 expresa conflicto de estado, unicidad o concurrencia. Cada condición funcional se detalla en su fila.

Fuera de las ejecuciones de la tabla: URI inexistente o que no satisface la restricción de ruta →

404 NOT_FOUND; URI reconocida con método no admitido → 405 METHOD_NOT_ALLOWED, con header Allow. HEAD no está publicado como operación y su respuesta no lleva Body por semántica HTTP. Estas respuestas de routing no agregan nuevos endpoints. El formato común de error se define en §6.

Los filtros de seguridad se ejecutan antes de la validación MVC: en una ruta protegida, un token ausente o inválido puede producir 401 INVALID_TOKEN antes de que se evalúe el Body. Un rol no permitido puede producir 403 FORBIDDEN antes de validar los datos. La ruta se resuelve antes de ejecutar esos filtros.

2.4 Headers Location de creación

Los once 201 emiten Location relativo al origen. L1–L11 son referencias de headers de response, no datos de entrada. Los valores se completan con los identificadores del resultado. Un destino no implica un endpoint GET si no aparece publicado en la tabla.

Tabla 3. Valores de Location de las once respuestas 201.

Referencia	Operación que devuelve 201	Response: Location
L1	POST /api/v1/sesiones	/api/v1/sesiones/{token}
L2	POST /api/v1/jugadores	/api/v1/usuarios/{id}
L3	POST /api/v1/usuarios	/api/v1/usuarios/{id}
L4	POST /api/v1/repertorios	/api/v1/repertorios/{skillId}
L5	POST /api/v1/skills	/api/v1/skills/{id}
L6	POST /api/v1/torneos	/api/v1/torneos/{id}
L7	POST /api/v1/torneos/{torneoId}/skills-obligatorias	/api/v1/torneos/{torneoId}/skills-obligatorias/{skillId}
L8	POST /api/v1/torneos/{torneoId}/rondas	/api/v1/torneos/{torneoId}/rondas/{numero}
L9	POST /api/v1/inscripciones	/api/v1/inscripciones/{id}
L10	POST /api/v1/batallas/{id}/lanzamientos	/api/v1/batallas/{id}/lanzamientos
L11	POST /api/v1/batallas/{id}/fallos	/api/v1/batallas/{id}

L1 identifica una sesión, pero no existe GET /sesiones/{token}. L2 y L3 apuntan al usuario creado; GET /usuarios/{id} requiere Administrador. L4 y L7 identifican asociaciones cuyo método individual publicado es DELETE. L10 apunta a la colección de lanzamientos; la respuesta del POST es S13 y no incluye id propio. L11 apunta a la batalla actualizada con el fallo. Estas limitaciones se describen como comportamiento efectivo, sin inventar rutas de lectura.

3. Criterios REST: por qué la API los cumple

REST es un estilo de arquitectura, definido por Fielding, que se presenta en el módulo M1 del curso [3]. Usar HTTP y JSON no alcanza para que una API sea REST: tiene que respetar un conjunto de restricciones. Este capítulo recorre cada restricción y cada práctica de diseño del curso, explica por qué la API de FarmeORT la cumple y da un ejemplo tomado de la propia API.

3.1 Las restricciones de REST

Cliente-servidor. La restricción pide separar la interfaz de usuario del almacenamiento y la lógica, para que cada lado evolucione por su cuenta. FarmeORT no tiene interfaz: el cliente solo conoce las URI y los JSON de la tabla central. No sabe que existen servicios, repositorios ni una base SQL Server. Prueba de eso es que toda la API se ejercita desde la colección de Postman, y que un frontend futuro usaría exactamente los mismos pedidos, sin cambiar el backend.

Sin estado (stateless). Cada pedido tiene que traer todo lo necesario para atenderlo, sin depender de lo que el servidor recuerde de pedidos anteriores. En FarmeORT cada pedido protegido trae su token en `Authorization`; el servidor no usa sesiones de ASP.NET, cookies ni memoria por cliente. La sesión es una fila en la base que se consulta en cada pedido, igual que un torneo o una skill. Por eso, si el servidor se reinicia entre dos pedidos, el segundo funciona igual, y cualquier instancia de la API podría atenderlo. La paginación sigue el mismo criterio: el cliente manda `limit` y `offset` en cada pedido, y el servidor no guarda un cursor.

Cacheable. Las respuestas tienen que poder declararse reutilizables. En FarmeORT ningún GET modifica datos: todos los cambios entran por POST, PUT, PATCH o DELETE. Eso hace que las respuestas de GET sean cacheables por su semántica; por ejemplo, un proxy podría guardar `GET /api/v1/rankings`, que es público. La API no envía `Cache-Control` ni `ETag`, por una decisión consciente: el ranking y los torneos cambian con cada fallo e inscripción, y en un torneo en curso importa ver el dato actual.

Interfaz uniforme. Es la restricción central y tiene cuatro partes:

- **Identificación de recursos.** Cada recurso se identifica con una URI: el torneo 4 es `/api/v1/torneos/4` y su ronda 2, `/api/v1/torneos/4/rondas/2`.
- **Manipulación mediante representaciones.** El cliente cambia un recurso enviando su representación JSON: un `PUT /api/v1/torneos/4` con el `TorneoRequest` completo reemplaza los datos editables del torneo.
- **Mensajes autodescriptivos.** Cada respuesta se entiende sola: el método dice qué se pidió, el status dice qué pasó, `Content-Type: application/json` dice cómo leer el cuerpo, y el `errorCode` dice la causa exacta de un rechazo.
- **Hipermedia (HATEOAS), en forma parcial.** Cada alta responde 201 con el header `Location`, que lleva a la URI del recurso creado; por ejemplo, `Location: /api/v1/inscripciones/12`. Fuera de `Location`, las respuestas no incluyen enlaces de navegación: los clientes de esta API se conocen de antemano y la tabla central describe todas las operaciones.

Sistema en capas. El cliente no puede distinguir si habla directamente con la API o con un

intermediario. La API se podría publicar detrás de un proxy inverso o de un balanceador sin cambiar ninguna URI ni ningún JSON. Hacia adentro, la WebAPI solo conoce las interfaces de los servicios y los DTOs, y nunca expone entidades del dominio: el contrato no cambia si cambia la persistencia.

Código bajo demanda. Es la única restricción opcional: permite que el servidor envíe código ejecutable al cliente. FarmeORT no la usa porque devuelve solo datos JSON.

3.2 Prácticas de diseño del curso

1. Sustantivos, no verbos. Las URI nombran recursos y el verbo HTTP dice la acción: `POST /api/v1/torneos` crea un torneo, sin rutas como `/crearTorneo`. Las operaciones que no son un alta, una lectura, un reemplazo o una baja se modelan como un sustantivo: cancelar un torneo es crear una cancelación (`POST /api/v1/torneos/4/cancelaciones`), y fallar una batalla es crear un fallo (`POST /api/v1/batallas/10/fallos`).

2. Plural y consistencia. Todas las colecciones van en plural y los campos en camelCase. En la revisión del 05/10/2026 el docente marcó que `/ranking` era el único recurso en singular, y pasó a `/rankings` (D-69).

3. Jerarquía de hasta tres niveles. Un subrecurso cuelga de una instancia identificada, como `/api/v1/torneos/4/rondas/2` o `/api/v1/batallas/10/lanzamientos`, y ninguna ruta pasa de tres niveles. En la misma revisión se eliminó `/jugadores/actual/repertorio`, porque `actual` no es un identificador y la ruta se pasaba de nivel: ahora es `/api/v1/repertorios` y el jugador sale del token.

4. La complejidad va después del signo de pregunta. Los filtros y la paginación van en la query y no en la ruta: `GET /api/v1/skills?nombre=six&categoria=Gestual&activa=true&limit=20&offset=0`. Un valor fuera de rango no se rechaza, para no trancar al usuario: `limit=500` usa el tope de 100 y un `offset` negativo usa 0. Un tipo mal formado, como `limit=abc`, sí da 400.

5. Versionado en la URI. Todas las rutas empiezan con `/api/v1`. Si un cambio rompiera el contrato, se publicaría `/api/v2` y los clientes de la versión 1 seguirían funcionando.

6. Cada verbo con su semántica. GET lee sin modificar; POST crea; PUT reemplaza todos los datos editables (`PUT /api/v1/usuarios/{id}` exige todos los campos); PATCH cambia una parte (`PATCH /api/v1/skills/{id}` con `{ "activa": false }` solo desactiva la skill), y DELETE quita (`DELETE /api/v1/repertorios/{skillId}`).

7. Un status por causa. El status permite que el cliente reaccione sin leer el mensaje. Un alta responde 201 con `Location`, y una baja, 204 sin cuerpo. Un dato mal formado es 400 y una regla de negocio violada es 409. Por ejemplo, crear un torneo con cupo 6 da 400 `INVALID_DATA`, porque el dato en sí es inválido (no es potencia de 2), mientras que inscribirse en un torneo con el cupo lleno da 409 `TOURNAMENT_FULL`, porque el dato es válido pero el estado del torneo no lo admite. Sin token o con un token inválido es 401; con un rol que no corresponde, 403; un recurso que no existe, 404. Nunca se devuelve 200 con un error en el cuerpo, ni 500 por culpa del cliente.

8. Errores útiles. Todos los errores tienen el mismo formato, así que el cliente los procesa con

un solo código. Por ejemplo, inscribirse en un torneo completo responde:

```
HTTP/1.1 409 Conflict

{ "userMessage": "El torneo ya alcanzó su cupo de inscripciones aceptadas.",
  "errorCode": "TOURNAMENT_FULL", "moreInfo": null, "detalles": null }
```

`userMessage` se puede mostrar al usuario, `errorCode` permite al cliente decidir qué hacer, y `detalles` lista los campos que fallaron en una validación. Un error inesperado responde 500 con un mensaje fijo, sin trazas, consultas SQL ni rutas internas, que le darían información a un atacante.

9. La identidad sale del token, no del pedido. Ninguna operación recibe el id del usuario actual en la ruta, la query o el body. El registro no recibe un rol: el servidor asigna Jugador. El repertorio, la inscripción y el lanzamiento usan el usuario del token (sección 2.1). Por eso la ruta del repertorio es `/api/v1/repertorios` y no `/jugadores/7/repertorio`.

10. Seguridad e idempotencia. GET es seguro: no cambia nada. PUT, PATCH y DELETE son idempotentes: repetir el pedido deja el mismo estado. Borrar dos veces el mismo usuario lo deja dado de baja; el segundo intento responde 404, pero el estado no cambia. POST no es idempotente, y la API lo refleja: dos logins crean dos sesiones con tokens distintos.

11. JSON como formato de intercambio. Los cuerpos se envían y se reciben en JSON. Un cuerpo en otro formato responde 415 `UNSUPPORTED_MEDIA_TYPE`, y un JSON mal formado, 400 `VALIDATION_ERROR`.

4. Tabla central de las operaciones vigentes

Cada fila describe un método publicado. — indica ausencia de query, Body o ResponseBody. R01–R11 y S01–S14 remiten a JSON compartidos en el capítulo inmediatamente posterior; Sxx[] es un array directo del objeto Sxx. L1–L11 están definidos en la Tabla 3. Content-Type y Accept tienen la obligatoriedad indicada en §2.2. Los errores retornan ErrorResponse (§6). Cada status de error se muestra junto a su errorCode (por ejemplo, 403 FORBIDDEN), con el mismo valor que devuelve el campo errorCode y que lista la Tabla 31; las respuestas 2xx muestran su frase HTTP porque no llevan errorCode. Cada query param declara su tipo de dato (string, int32, bool o date con formato YYYY-MM-DD); los requeridos se marcan con [REQ].

* Generar rondas y finalizar el torneo también los puede pedir un Administrador. La letra [1] asigna estas operaciones al Organizador y no dice nada del Administrador; el equipo lo habilitó para que un torneo no quede trabado si el organizador asignado no está disponible (supuesto registrado en la bitácora, D-63). Un Organizador que no es el asignado recibe 403 NOT_TOURNAMENT_ORGANIZER.

Tabla 4. Contrato HTTP integrado de las 38 operaciones. La cabecera se repite al continuar.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
Sesión							
POST	/api/v1/sesiones	Anónimo	Content-Type: application/json Accept: application/json Response: Location: L1	—	JSON: R01	JSON: S01	**201 Created**: sesión creada; devuelve token, usuarioId y rol. **400 VALIDATION_ERROR**: email, contraseña o JSON inválidos. **401 INVALID_CREDENTIALS**: credenciales incorrectas o usuario dado de baja. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.
DELETE	/api/v1/sesiones/{token}	Autenticado	Authorization: Bearer {token}	—	—	—	**204 No Content**: sesión propia cerrada. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 NOT_SESSION_OWNER**: la sesión objetivo pertenece a otro usuario. **404 SESSION_NOT_FOUND**: sesión objetivo inexistente. **404 NOT_FOUND**: token de ruta que no es GUID. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
Jugador							
POST	/api/v1/jugadores	Anónimo	Content-Type: application/json Accept: application/json Response: Location: L2	—	JSON: R02	JSON: S02	**201 Created**: jugador creado con ELO 1200 y repertorio vacío. **400 VALIDATION_ERROR**: campos de registro inválidos, email >254 o JSON inválido. **400 INVALID_DATA**: fecha de nacimiento futura o edad menor de 13 años. **409 EMAIL_ALREADY_EXISTS**: email ya utilizado. **409 NICKNAME_ALREADY_EXISTS**: nickname ya utilizado. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.
Usuario							
POST	/api/v1/usuarios	Administrador	Authorization: Bearer {token} Content-Type: application/json Accept: application/json Response: Location: L3	—	JSON: R03	JSON: S02	**201 Created**: usuario creado con el rol indicado. **400 VALIDATION_ERROR**: campos, rol, email >254 o JSON inválidos. **400 INVALID_DATA**: fecha de nacimiento futura o edad menor de 13 años. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **409 EMAIL_ALREADY_EXISTS**: email ya utilizado. **409 NICKNAME_ALREADY_EXISTS**: nickname ya utilizado. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.
GET	/api/v1/usuarios	Administrador	Authorization: Bearer {token} Accept: application/json	**nombre (string)**: contiene el texto, sin distinguir mayúsculas. **apellido (string)**: contiene el texto, sin distinguir mayúsculas. **nickname (string)**: contiene el texto, sin distinguir mayúsculas. **limit (int32)**: default 20; >100 usa 100; <1 usa 20. **offset (int32)**: default 0; negativo usa 0.	—	JSON: S02[]	**200 OK**: usuarios activos filtrados y paginados; [] si no hay coincidencias. **400 VALIDATION_ERROR**: tipo de query param no convertible (p. ej. limit=abc). **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
GET	/api/v1/usuarios/{id}	Administrador	Authorization: Bearer {token} Accept: application/json	—	—	JSON: S02	**200 OK**: usuario activo obtenido. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 USER_NOT_FOUND**: usuario inexistente o dado de baja. **404 NOT_FOUND**: id de ruta incompatible con int32. **500 INTERNAL_ERROR**: error interno inesperado.
PUT	/api/v1/usuarios/{id}	Administrador	Authorization: Bearer {token} Content-Type: application/json Accept: application/json	—	JSON: R03	JSON: S02	**200 OK**: datos completos del usuario actualizados. **400 VALIDATION_ERROR**: campos, rol, email >254 o JSON inválidos. **400 INVALID_DATA**: fecha de nacimiento futura o edad menor de 13 años. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 USER_NOT_FOUND**: usuario inexistente o dado de baja. **409 SELF_OPERATION_NOT_ALLOWED**: el administrador intenta modificarse a sí mismo. **409 EMAIL_ALREADY_EXISTS**: email de otro usuario. **409 NICKNAME_ALREADY_EXISTS**: nickname de otro usuario. **409 USER_ORGANIZES_ACTIVE_TOURNAMENT**: cambio de rol con un torneo organizado vigente. **409 USER_HAS_UNRESOLVED_BATTLES**: cambio de rol con batallas sin resolver. **409 USER_HAS_ACTIVE_REGISTRATION**: cambio de rol con inscripción vigente. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
DELETE	/api/v1/usuarios/{id}	Administrador	Authorization: Bearer {token}	—	—	—	**204 No Content**: baja lógica y revocación de todas sus sesiones. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 USER_NOT_FOUND**: usuario inexistente o dado de baja. **404 NOT_FOUND**: id de ruta incompatible con int32. **409 SELF_OPERATION_NOT_ALLOWED**: el administrador intenta darse de baja. **409 USER_ORGANIZES_ACTIVE_TOURNAMENT**: organiza un torneo no Finalizado ni Cancelado. **409 USER_HAS_UNRESOLVED_BATTLES**: tiene batallas Pendientes o EnArbitraje. **409 USER_HAS_ACTIVE_REGISTRATION**: tiene una inscripción vigente. **500 INTERNAL_ERROR**: error interno inesperado.
Repertorio							
GET	/api/v1/repertorios	Jugador	Authorization: Bearer {token} Accept: application/json	—	—	JSON: S03[]	**200 OK**: skills declaradas del jugador del Bearer, ordenadas por skillId. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 USER_NOT_FOUND**: jugador ya inexistente al consultar. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
POST	/api/v1/repertorios	Jugador	Authorization: Bearer {token} Content-Type: application/json Accept: application/json Response: Location: L4	—	{"skillId":3, "nivel":4} **skillId [REQ]** (int32) **nivel [REQ]** (int32 1..5)	JSON: S03	**201 Created**: skill declarada en el repertorio del jugador. **400 VALIDATION_ERROR**: skillId/nivel omitidos, nivel fuera de 1..5 o JSON inválido. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 USER_NOT_FOUND**: jugador inexistente. **404 SKILL_NOT_FOUND**: skill inexistente. **409 SKILL_INACTIVE**: no es posible utilizar la skill porque está inactiva. **409 SKILL_ALREADY_DECLARED**: skill ya declarada, incluso por declaración concurrente. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.
DELETE	/api/v1/repertorios/{skillId}	Jugador	Authorization: Bearer {token}	—	—	—	**204 No Content**: skill quitada del repertorio propio. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 USER_NOT_FOUND**: jugador inexistente. **404 DECLARED_SKILL_NOT_FOUND**: la skill no está en el repertorio. **404 NOT_FOUND**: id de ruta incompatible con int32. **409 MANDATORY_SKILL_IN_USE**: obligatoria de un torneo no Finalizado ni Cancelado con inscripción Aceptada propia. **500 INTERNAL_ERROR**: error interno inesperado.
Skill							
GET	/api/v1/skills	Autenticado	Authorization: Bearer {token} Accept: application/json	**categoria (string)**: Gestual | Vocal | Baile | Meme. **dificultad (int32)**: 1..5; fuera de rango se ignora. **activa (bool)**: true | false. **nombre (string)**: contiene el texto, sin distinguir mayúsculas. **limit (int32)**: default 20; >100 usa 100; <1 usa 20. **offset (int32)**: default 0; negativo usa 0.	—	JSON: S04[]	**200 OK**: catálogo filtrado, paginado y ordenado por id. **400 VALIDATION_ERROR**: dificultad, activa, limit u offset no convertible. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
GET	/api/v1/skills/{id}	Autenticado	Authorization: Bearer {token} Accept: application/json	—	—	JSON: S04	**200 OK**: skill obtenida, también si está inactiva. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **404 SKILL_NOT_FOUND**: skill inexistente. **404 NOT_FOUND**: id de ruta incompatible con int32. **500 INTERNAL_ERROR**: error interno inesperado.
POST	/api/v1/skills	Administrador	Authorization: Bearer {token} Content-Type: application/json Accept: application/json Response: Location: L5	—	JSON: R04	JSON: S04	**201 Created**: skill creada inicialmente activa. **400 VALIDATION_ERROR**: campos de skill inválidos o JSON inválido. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **409 SKILL_CODE_ALREADY_EXISTS**: código de skill ya utilizado. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.
PUT	/api/v1/skills/{id}	Administrador	Authorization: Bearer {token} Content-Type: application/json Accept: application/json	—	JSON: R04	JSON: S04	**200 OK**: datos completos de la skill actualizados; conserva activa. **400 VALIDATION_ERROR**: campos de skill inválidos o JSON inválido. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 SKILL_NOT_FOUND**: skill inexistente. **409 SKILL_CODE_ALREADY_EXISTS**: el código pertenece a otra skill. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
PATCH	/api/v1/skills/{id}	Administrador	Authorization: Bearer {token} Content-Type: application/json Accept: application/json	—	{"activa": false} **activa [REQ]** (bool)	JSON: S04	**200 OK**: activa/inactiva la skill; no altera usos anteriores. **400 VALIDATION_ERROR**: activa ausente, null o body inválido. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 SKILL_NOT_FOUND**: skill inexistente. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.
Torneo							
GET	/api/v1/torneos	Todos	Authorization: Bearer {token} (opcional) Accept: application/json	**estado (string)**: Abierto | EnCurso | Finalizado | Cancelado. **fechaDesde (date YYYY-MM-DD)**: fechaInicio >= valor, inclusiva. **fechaHasta (date YYYY-MM-DD)**: fechaInicio <= valor, inclusiva. **nombre (string)**: contiene el texto, sin distinguir mayúsculas. **skillObligatoriaId (int32)**: asociación exacta. **limit (int32)**: default 20; >100 usa 100; <1 usa 20. **offset (int32)**: default 0; negativo usa 0.	—	JSON: S05[]	**200 OK**: torneos visibles filtrados, ordenados por id y paginados. **400 VALIDATION_ERROR**: fecha, skillObligatoriaId, limit u offset no convertible. **401 INVALID_TOKEN**: Bearer enviado inválido; sin header se permite anónimo. **500 INTERNAL_ERROR**: error interno inesperado.
GET	/api/v1/torneos/{id}	Todos	Authorization: Bearer {token} (opcional) Accept: application/json	—	—	JSON: S06	**200 OK**: detalle con obligatorias, jugadores Aceptados y cuadro completo. **401 INVALID_TOKEN**: Bearer enviado inválido; sin header se permite anónimo. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente o Cancelado para anónimo. **404 NOT_FOUND**: id de ruta incompatible con int32. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
POST	/api/v1/torneos	Administrador	Authorization: Bearer {token} Content-Type: application/json Accept: application/json Response: Location: L6	—	JSON: R05	JSON: S05	**201 Created**: torneo creado Abierto y sin inscripciones. **400 VALIDATION_ERROR**: campos obligatorios, formato o JSON inválidos. **400 INVALID_DATA**: cupo no potencia de 2, fechas inválidas o usuario asignado sin rol Organizador. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 USER_NOT_FOUND**: organizador inexistente o dado de baja. **409 TOURNAMENT_CODE_ALREADY_EXISTS**: código de torneo ocupado. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.
PUT	/api/v1/torneos/{id}	Administrador	Authorization: Bearer {token} Content-Type: application/json Accept: application/json	—	JSON: R05	JSON: S05	**200 OK**: torneo Abierto actualizado sin cancelar inscripciones. **400 VALIDATION_ERROR**: campos obligatorios, formato o JSON inválidos. **400 INVALID_DATA**: cupo no potencia de 2, fechas inválidas o usuario asignado sin rol Organizador. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente. **404 USER_NOT_FOUND**: organizador inexistente o dado de baja. **409 TOURNAMENT_NOT_MODIFIABLE**: el torneo no está Abierto. **409 TOURNAMENT_CODE_ALREADY_EXISTS**: código ocupado por otro torneo. **409 TOURNAMENT_CAPACITY_BELOW_REGISTRATIONS**: cupo menor que las inscripciones Aceptadas. **409 TOURNAMENT_CHANGED**: el torneo cambió durante el pedido. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
Cancelación de torneo							
POST	/api/v1/torneos/{id}/cancelaciones	Administrador	Authorization: Bearer {token} Accept: application/json	—	—	JSON: S05	**200 OK**: cancela torneo, batallas sin resolver e inscripciones Aceptadas, atómicamente. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente. **404 NOT_FOUND**: id de ruta incompatible con int32. **409 TOURNAMENT_NOT_CANCELLABLE**: torneo ya Finalizado o Cancelado. **409 TOURNAMENT_CHANGED**: el torneo cambió durante el pedido. **409 BATTLE_CHANGED**: la batalla o un ELO cambió durante el pedido. **500 INTERNAL_ERROR**: error interno inesperado.
Finalización de torneo							
POST	/api/v1/torneos/{torneoId}/finalizaciones	Organizador del torneo / Administrador *	Authorization: Bearer {token} Accept: application/json	—	—	JSON: S07	**200 OK**: torneo Finalizado con campeón registrado. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no permitido. **403 NOT_TOURNAMENT_ORGANIZER**: Organizador no asignado al torneo. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente. **404 NOT_FOUND**: id de ruta incompatible con int32. **409 TOURNAMENT_NOT_IN_PROGRESS**: el torneo no está EnCurso. **409 TOURNAMENT_HAS_UNRESOLVED_BATTLES**: quedan batallas sin resolver. **409 TOURNAMENT_WITHOUT_CHAMPION**: no queda exactamente un jugador Aceptado. **409 TOURNAMENT_CHANGED**: el torneo cambió durante el pedido. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
Skill obligatoria							
GET	/api/v1/torneos/{torneoId}/skills-obligatorias	Todos	Authorization: Bearer {token} (opcional) Accept: application/json	—	—	JSON: S04[]	**200 OK**: skills obligatorias del torneo, ordenadas por id. **401 INVALID_TOKEN**: Bearer enviado inválido; sin header se permite anónimo. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente o Cancelado para anónimo. **404 NOT_FOUND**: id de ruta incompatible con int32. **500 INTERNAL_ERROR**: error interno inesperado.
POST	/api/v1/torneos/{torneoId}/skills-obligatorias	Organizador del torneo	Authorization: Bearer {token} Content-Type: application/json Accept: application/json Response: Location: L7	—	{"skillId":3} **skillId [REQ]** (int32)	JSON: S04	**201 Created**: skill asociada como obligatoria al torneo. **400 VALIDATION_ERROR**: skillId omitido o JSON inválido. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no Organizador. **403 NOT_TOURNAMENT_ORGANIZER**: Organizador no asignado al torneo. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente. **404 SKILL_NOT_FOUND**: skill inexistente. **409 TOURNAMENT_NOT_MODIFIABLE**: el torneo no está Abierto. **409 SKILL_INACTIVE**: no es posible utilizar la skill porque está inactiva. **409 MANDATORY_SKILL_ALREADY_ADDED**: la skill ya es obligatoria en el torneo. **409 MANDATORY_SKILLS_LIMIT_REACHED**: el torneo ya tiene 2 obligatorias. **409 PLAYERS_MISSING_MANDATORY_SKILL**: hay Aceptados sin la skill; detalles lista sus nicknames. **409 TOURNAMENT_CHANGED**: el torneo cambió durante el pedido. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
DELETE	/api/v1/torneos/{torneoId}/skills-obligatorias/{skillId}	Organizador del torneo	Authorization: Bearer {token}	—	—	—	**204 No Content**: asociación obligatoria eliminada. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no Organizador. **403 NOT_TOURNAMENT_ORGANIZER**: Organizador no asignado al torneo. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente. **404 MANDATORY_SKILL_NOT_FOUND**: la skill no es obligatoria en el torneo. **404 NOT_FOUND**: id de ruta incompatible con int32. **409 TOURNAMENT_NOT_MODIFIABLE**: el torneo no está Abierto. **409 TOURNAMENT_CHANGED**: el torneo cambió durante el pedido. **500 INTERNAL_ERROR**: error interno inesperado.
Ronda							
POST	/api/v1/torneos/{torneoId}/rondas	Organizador del torneo / Administrador *	Authorization: Bearer {token} Accept: application/json Response: Location: L8	—	—	JSON: S08	**201 Created**: ronda y batallas creadas; la primera ronda inicia el torneo. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no permitido. **403 NOT_TOURNAMENT_ORGANIZER**: Organizador no asignado al torneo. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente. **404 NOT_FOUND**: id de ruta incompatible con int32. **409 TOURNAMENT_NOT_OPEN**: primera ronda con el torneo no Abierto. **409 TOURNAMENT_NOT_FULL**: primera ronda sin el cupo completo. **409 ROUND_NOT_RESOLVED**: la ronda anterior tiene batallas sin resolver. **409 TOURNAMENT_READY_TO_FINISH**: queda menos de 2 Aceptados: corresponde finalizar. **409 TOURNAMENT_CHANGED**: lista temporal impar o versión del torneo cambiada. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
GET	/api/v1/torneos/{torneoId}/rondas	Todos	Authorization: Bearer {token} (opcional) Accept: application/json	—	—	JSON: S08[]	**200 OK**: rondas generadas con sus batallas; [] si no hay. **401 INVALID_TOKEN**: Bearer enviado inválido; sin header se permite anónimo. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente o Cancelado para anónimo. **404 NOT_FOUND**: id de ruta incompatible con int32. **500 INTERNAL_ERROR**: error interno inesperado.
GET	/api/v1/torneos/{torneoId}/rondas/{numero}	Todos	Authorization: Bearer {token} (opcional) Accept: application/json	—	—	JSON: S08	**200 OK**: ronda del número solicitado con sus batallas. **401 INVALID_TOKEN**: Bearer enviado inválido; sin header se permite anónimo. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente o Cancelado para anónimo. **404 ROUND_NOT_FOUND**: el torneo no tiene una ronda con ese número. **404 NOT_FOUND**: id de ruta incompatible con int32. **500 INTERNAL_ERROR**: error interno inesperado.
Inscripción							
POST	/api/v1/inscripciones	Jugador	Authorization: Bearer {token} Content-Type: application/json Accept: application/json Response: Location: L9	—	{"torneoId":4} **torneoId [REQ]** (int32)	JSON: S09	**201 Created**: inscripción propia creada Aceptada. **400 VALIDATION_ERROR**: torneoId omitido o JSON inválido. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente. **404 USER_NOT_FOUND**: jugador inexistente. **409 TOURNAMENT_NOT_OPEN**: el torneo no está Abierto. **409 ALREADY_REGISTERED**: ya tiene una inscripción Aceptada en el torneo. **409 MISSING_MANDATORY_SKILLS**: faltan obligatorias; detalles lista sus códigos. **409 TOURNAMENT_FULL**: cupo de Aceptadas completo. **409 TOURNAMENT_CHANGED**: el torneo cambió durante el pedido. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
GET	/api/v1/inscripciones	Autenticado	Authorization: Bearer {token} Accept: application/json	**torneoId (int32)**: igualdad exacta. **estado (string)**: Aceptada | Cancelada | Eliminada. **fechaDesde (date YYYY-MM-DD)**: día de inscripción, inclusivo. **fechaHasta (date YYYY-MM-DD)**: día de inscripción, inclusivo. **limit (int32)**: default 20; >100 usa 100; <1 usa 20. **offset (int32)**: default 0; negativo usa 0.	—	JSON: S09[]	**200 OK**: inscripciones visibles: Jugador propias, Organizador sus torneos, Administrador todas. **400 VALIDATION_ERROR**: torneoId, fechas, limit u offset no convertible. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **500 INTERNAL_ERROR**: error interno inesperado.
GET	/api/v1/inscripciones/{id}	Autenticado	Authorization: Bearer {token} Accept: application/json	—	—	JSON: S09	**200 OK**: inscripción visible obtenida. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 NOT_REGISTRATION_OWNER**: Jugador no propietario. **403 NOT_TOURNAMENT_ORGANIZER**: Organizador no asignado al torneo. **404 REGISTRATION_NOT_FOUND**: inscripción inexistente. **404 NOT_FOUND**: id de ruta incompatible con int32. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
Cancelación de inscripción							
POST	/api/v1/inscripciones/{id}/cancelaciones	Jugador propietario	Authorization: Bearer {token} Accept: application/json	—	—	JSON: S09	**200 OK**: inscripción propia Aceptada pasa a Cancelada. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no Jugador. **403 NOT_REGISTRATION_OWNER**: la inscripción no es propia. **404 REGISTRATION_NOT_FOUND**: inscripción inexistente. **404 NOT_FOUND**: id de ruta incompatible con int32. **409 TOURNAMENT_NOT_OPEN**: el torneo no está Abierto. **409 REGISTRATION_NOT_ACTIVE**: la inscripción ya no está Aceptada. **409 TOURNAMENT_CHANGED**: el torneo cambió durante el pedido. **500 INTERNAL_ERROR**: error interno inesperado.
Batalla							
GET	/api/v1/batallas	Jugador / Organizador / Administrador	Authorization: Bearer {token} Accept: application/json	**torneoId [REQ] (int32)**: > 0. **estado (string)**: Pendiente | EnArbitraje | Finalizada | Cancelada. **ronda (int32)**: > 0; <= 0 se ignora. **jugadorId (int32)**: > 0; <= 0 se ignora. **limit (int32)**: default 20; >100 usa 100; <1 usa 20. **offset (int32)**: default 0; negativo usa 0.	—	JSON: S10[]	**200 OK**: batallas visibles, ordenadas por ronda/id y paginadas. **400 VALIDATION_ERROR**: torneoId ausente o no positivo; tipo de query no convertible. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 PLAYER_NOT_IN_TOURNAMENT**: Jugador sin inscripción en el torneo. **403 NOT_TOURNAMENT_ORGANIZER**: Organizador no asignado al torneo. **404 TOURNAMENT_NOT_FOUND**: torneo inexistente. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
GET	/api/v1/batallas/{id}	Jugador / Organizador / Administrador	Authorization: Bearer {token} Accept: application/json	—	—	JSON: S11	**200 OK**: detalle visible con lanzamientos ordenados y resultado, si existe. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 PLAYER_NOT_IN_TOURNAMENT**: Jugador sin inscripción en el torneo. **403 NOT_TOURNAMENT_ORGANIZER**: Organizador no asignado al torneo. **404 BATTLE_NOT_FOUND**: batalla inexistente. **404 NOT_FOUND**: id de ruta incompatible con int32. **500 INTERNAL_ERROR**: error interno inesperado.
Lanzamiento							
GET	/api/v1/batallas/{id}/lanzamientos	Jugador / Organizador / Administrador	Authorization: Bearer {token} Accept: application/json	—	—	JSON: S12[]	**200 OK**: historial visible de lanzamientos por turno; [] si no hay. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 PLAYER_NOT_IN_TOURNAMENT**: Jugador sin inscripción en el torneo. **403 NOT_TOURNAMENT_ORGANIZER**: Organizador no asignado al torneo. **404 BATTLE_NOT_FOUND**: batalla inexistente. **404 NOT_FOUND**: id de ruta incompatible con int32. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
POST	/api/v1/batallas/{id}/lanzamientos	Jugador	Authorization: Bearer {token} Content-Type: application/json Accept: application/json Response: Location: L10	—	{"skillId":3} **skillId [REQ]** (int32 >= 1)	JSON: S13	**201 Created**: turno registrado; el octavo lanzamiento pasa a EnArbitraje. **400 VALIDATION_ERROR**: skillId ausente o no positivo; JSON inválido. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no habilitado para esta operación. **404 BATTLE_NOT_FOUND**: batalla inexistente. **404 USER_NOT_FOUND**: jugador inexistente. **404 SKILL_NOT_FOUND**: skill inexistente. **409 BATTLE_NOT_PENDING**: la batalla no está Pendiente. **409 PLAYER_NOT_IN_BATTLE**: el jugador no participa de la batalla. **409 NOT_PLAYER_TURN**: no es su turno. **409 SKILL_NOT_IN_REPERTOIRE**: la skill no está en su repertorio. **409 SKILL_ALREADY_THROWN**: skill ya lanzada en la batalla. **409 MANDATORY_SKILL_PENDING**: debe lanzar una obligatoria pendiente. **409 BATTLE_CHANGED**: la batalla o un ELO cambió durante el pedido. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.

Verbo	URI	Acceso	Headers	Query params	Body	Response body	Status codes y errores
Fallo							
POST	/api/v1/batallas/{id}/fallos	Organizador del torneo	Authorization: Bearer {token} Content-Type: application/json Accept: application/json Response: Location: L11	—	JSON: R11	JSON: S11	**201 Created**: batalla Finalizada; ELO actualizado y perdedor Eliminado, atómicamente. **400 VALIDATION_ERROR**: ganadorId ausente o no positivo; comentario fuera de 10..500; JSON inválido. **401 INVALID_TOKEN**: Bearer ausente, inválido o revocado. **403 FORBIDDEN**: rol no Organizador. **403 NOT_TOURNAMENT_ORGANIZER**: Organizador no asignado al torneo. **404 BATTLE_NOT_FOUND**: batalla inexistente. **404 NOT_FOUND**: id de ruta incompatible con int32. **409 BATTLE_NOT_JUDGEABLE**: batalla fuera de Pendiente/EnArbitraje. **409 PLAYER_NOT_IN_BATTLE**: el ganador no participa de la batalla. **409 REGISTRATION_NOT_ACTIVE**: la inscripción del perdedor no está Aceptada. **409 BATTLE_CHANGED**: la batalla o un ELO cambió durante el pedido. **415 UNSUPPORTED_MEDIA_TYPE**: tipo de contenido del body no admitido. **500 INTERNAL_ERROR**: error interno inesperado.
Ranking							
GET	/api/v1/rankings	Todos	Authorization: Bearer {token} (opcional) Accept: application/json	**cantidad (int32)**: default 10; >100 usa 100; <1 usa 10. **torneoId (int32)**: valida existencia; selecciona participantes Aceptados/Eliminados sin baja y con rol Jugador.	—	JSON: S14[]	**200 OK**: ranking por ELO DESC e id ASC; ganadas/perdidas globales incluso con torneoId. **400 VALIDATION_ERROR**: cantidad o torneoId no convertible. **401 INVALID_TOKEN**: Bearer enviado inválido; sin header se permite anónimo. **404 TOURNAMENT_NOT_FOUND**: torneo filtrado inexistente o Cancelado para anónimo. **500 INTERNAL_ERROR**: error interno inesperado.

5. JSON compartidos: requests y responses

Los siguientes ejemplos muestran propiedades y valores válidos. Los ids presuponen recursos existentes y una sesión con los permisos y el estado necesarios; las fechas de torneo se ajustan si el pedido se usa después de noviembre de 2026. No forman una secuencia única de ejecución. int32 es entero de 32 bits; bool, true/false; string, texto JSON. Un campo marcado con [REQ] no admite omisión ni null.

Todos los request bodies publicados son objetos JSON. Los campos de respuesta no marcados como nullable se serializan como valor u objeto; las colecciones son arrays no null. Fechas de nacimiento, torneo y declaración: YYYY-MM-DD. fecha de inscripción: DateTime ISO 8601 sin offset garantizado. fecha de lanzamiento: DateTimeOffset ISO 8601, registrado en UTC.

5.1 Requests R01–R11

Tabla 5. R01 — LoginRequest: JSON y contrato de campos.

```
{
"email": "ana@ejemplo.com",
"contrasena": "ClaveSegura2026!"
}
```

email [REQ] (string): no vacío, con formato de email. El login no impone el máximo 254.
contrasena [REQ] (string): no vacía. No vuelve a exigir la complejidad del alta: credenciales incorrectas producen 401 INVALID_CREDENTIALS.

Tabla 6. R02 — RegistroJugadorRequest: JSON y contrato de campos.

```
{
"nombre": "Ana",
"apellido": "Perez",
"email": "ana@ejemplo.com",
"nickname": "anaperez",
"contrasena": "ClaveSegura2026!",
"fechaNacimiento": "2000-01-15"
}
```

nombre [REQ] (string): no vacío, 1..50.
apellido [REQ] (string): 3..25.
email [REQ] (string): formato válido, máximo 254, único.
nickname [REQ] (string): ASCII alfanumérico sin espacios, 3..20, único.
contrasena [REQ] (string): 15..25; mayúscula ASCII, minúscula ASCII, dígito ASCII y símbolo.
fechaNacimiento [REQ] (date YYYY-MM-DD): no futura; edad >= 13 según la fecha local del servidor.
Servidor: asigna rol Jugador y ELO 1200; ninguno de los campos admite null.

Tabla 7. R03 — UsuarioAdminRequest: JSON y contrato de campos.

```
{
"nombre": "Ana",
"apellido": "Perez",
"email": "ana@ejemplo.com",
"nickname": "anaperez",
"contrasena": "ClaveSegura2026!",
"fechaNacimiento": "2000-01-15",
"rol": "Jugador"
}
```

nombre [REQ] (string): no vacío, 1..50.
apellido [REQ] (string): 3..25.
email [REQ] (string): formato válido, máximo 254, único.
nickname [REQ] (string): ASCII alfanumérico sin espacios, 3..20, único.
contrasena [REQ] (string): 15..25; mayúscula ASCII, minúscula ASCII, dígito ASCII y símbolo.
fechaNacimiento [REQ] (date YYYY-MM-DD): no futura; edad >= 13 según la fecha local del servidor.
rol [REQ] (string): Jugador | Organizador | Administrador, respetando mayúsculas.
PUT: exige todos los campos, incluida la contraseña; no admite modificación propia. Si cambia el rol, bloquea participación vigente. elo de la respuesta es null para otros roles.

Tabla 8. R04 — SkillRequest: JSON y contrato de campos.

```
{
"codigo": "SIXSEVEN",
"nombre": "Six-Seven",
"descripcion": "Movimiento gestual
de manos.",
"categoria": "Gestual",
"dificultad": 2
}
```

codigo [REQ] (string): ASCII alfanumérico 5..20, único.
nombre [REQ] (string): no vacío, 3..40.
descripcion [REQ] (string): no vacía, 20..300.
categoria [REQ] (string): Gestual | Vocal | Baile | Meme, exacta.
dificultad [REQ] (int32): 1..5.
Servidor: el alta deja activa=true. PUT conserva activa; PATCH la cambia.

Tabla 9. R05 — TorneoRequest: JSON y contrato de campos.

```
{
"codigo": "COPAAURA",
"nombre": "Copa Aura",
"descripcion": "Torneo de
habilidades de FarmeORT.",
"fechaInicio": "2026-11-01",
"fechaFin": "2026-11-02",
"cupo": 4,
"organizadorId": 2
}
```

codigo [REQ] (string): ASCII alfanumérico 5..20, único.
nombre [REQ] (string): no vacío, 5..60.
descripcion [REQ] (string): no vacía, 20..500.
fechaInicio [REQ] (date YYYY-MM-DD): al crear, >= hoy. PUT puede conservar una fechaInicio pasada si no cambia.
fechaFin [REQ] (date YYYY-MM-DD): posterior a fechaInicio.
cupo [REQ] (int32): 4 | 8 | 16 | 32 | 64, nunca menor que las inscripciones Aceptadas.
organizadorId [REQ] (int32): usuario existente con rol Organizador.
Fechas: informativas, sin transición automática de estado.

Tabla 10. R06 — SkillDeclaradaRequest: JSON y contrato de campos.

```
{
"skillId": 3,
"nivel": 4
}
```

skillId [REQ] (int32): sin Range positivo en el DTO: un id inexistente, incluso 0/negativo, resulta 404 SKILL_NOT_FOUND.
nivel [REQ] (int32): 1..5.
Reglas: la skill debe estar activa y no declarada; el jugador sale del Bearer.

Tabla 11. R07 — SkillEstadoRequest: JSON y contrato de campos.

```
{
"activa": false
}
```

activa [REQ] (bool): no null. Asignación de estado, no toggle: false inactiva; true activa.
Reglas: los usos anteriores se conservan.

Tabla 12. R08 — SkillObligatoriaRequest: JSON y contrato de campos.

```
{
"skillId": 3
}
```

skillId [REQ] (int32): sin Range en el DTO; inexistente resulta 404 SKILL_NOT_FOUND.
Reglas: Organizador asignado, torneo Abierto, skill activa, no repetida, máximo 2. Todos los jugadores Aceptados deben tenerla; si no, detalles enumera sus nicknames.

Tabla 13. R09 — InscripcionRequest: JSON y contrato de campos.

```
{
"torneoId": 4
}
```

torneoId [REQ] (int32): sin Range en el DTO; inexistente resulta 404 TOURNAMENT_NOT_FOUND.
Reglas: jugador del Bearer; torneo Abierto, sin Aceptada previa, obligatorias declaradas y cupo libre. detalles de obligatorias faltantes enumera sus códigos.

Tabla 14. R10 — LanzamientoRequest: JSON y contrato de campos.

```
{
"skillId": 3
}
```

skillId [REQ] (int32): 1..2147483647.
Reglas: batalla Pendiente; participante y turno propio; skill declarada y no repetida por ese jugador. Si obligatorias pendientes == turnos restantes, solo una obligatoria pendiente. Una skill ya declarada puede usarse aunque luego esté inactiva.

Tabla 15. R11 — FalloRequest: JSON y contrato de campos.

```
{
"ganadorId": 7,
"comentario": "Victoria por
abandono."
}
```

ganadorId [REQ] (int32): positivo y participante de la batalla.
comentario [REQ] (string): no vacío, 10..500.
Reglas: solo el Organizador asignado; batalla Pendiente (abandono) o EnArbitraje. Resultado, ELO y eliminación se confirman juntos.

5.2 Responses S01–S14

Tabla 16. S01 — SesionResponse: JSON y contrato de campos.

```
{
"token": "3f2b8c1e-5d4a-4e7b-9c6f-
1a2b3c4d5e6f",
"usuarioId": 7,
"rol": "Jugador"
}
```

token string GUID opaco; usuarioId int32; rol string. No contiene vencimiento ni JWT.

Tabla 17. S02 — UsuarioResponse: JSON y contrato de campos.

{
"id": 7,
"nombre": "Ana",
"apellido": "Perez",
"email": "ana@ejemplo.com",
"nickname": "anaperez",
"fechaNacimiento": "2000-01-15",
"rol": "Jugador",
"elo": 1200
}
	id int32; datos personales strings; fechaNacimiento fecha; rol string. elo int32 nullable: Jugador devuelve ELO, Organizador/Administrador devuelve null. Nunca retorna contraseña/hash.

Tabla 18. S03 — SkillDeclaradaResponse: JSON y contrato de campos.

{
"skillId": 3,
"codigo": "SIXSEVEN",
"nombre": "Six-Seven",
"categoria": "Gestual",
"activa": true,
"nivel": 4,
"fechaDeclaracion": "2026-10-07"
}
	skillId/nivel int32, activa bool, fechaDeclaracion fecha, demás strings. No expone jugadorId ni id propio de declaración.

Tabla 19. S04 — SkillResponse: JSON y contrato de campos.

{
"id": 3,
"codigo": "SIXSEVEN",
"nombre": "Six-Seven",
"descripcion": "Movimiento gestual de manos.",
"categoria": "Gestual",
"dificultad": 2,
"activa": true
}
	id/dificultad int32, activa bool, demás strings no null.

Tabla 20. S05 — TorneoResponse: JSON y contrato de campos.

{
"id": 4,
"codigo": "COPAAURA",
"nombre": "Copa Aura",
"descripcion": "Torneo de habilidades de FarmeORT.",
"fechaInicio": "2026-11-01",
"fechaFin": "2026-11-02",
"cupo": 4,
"organizadorId": 2,
"estado": "Abierto",
"inscripcionesAceptadas": 0,
"organizador": "Olga Perez"
}
	id/cupo/inscripcionesAceptadas/organizadorId int32; fechas tipo fecha; resto strings. organizador es nombre y apellido. No expone campeón ni Version.

Tabla 21. S06 — TorneoDetalleResponse: JSON y contrato de campos.


```
{
"id": 4,
"codigo": "COPAAURA",
"nombre": "Copa Aura",
"descripcion": "Torneo de habilidades
de FarmeORT.",
"fechaInicio": "2026-11-01",
"fechaFin": "2026-11-02",
"cupo": 4,
"organizadorId": 2,
"estado": "Abierto",
"inscripcionesAceptadas": 0,
"organizador": "Olga Perez",
"skillsObligatorias": [
{
"id": 3,
"codigo": "SIXSEVEN",
"nombre": "Six-Seven",
"descripcion": "Movimiento gestual de
manos.",
"categoria": "Gestual",
"dificultad": 2,
"activa": true
}
],
"jugadoresAceptados": [],
"rondas": []
}
```


Todas las propiedades S05, más skillsObligatorias array de S04 (0..2), jugadoresAceptados array de {jugadorId:int32,nickname:string,elo:int32}, rondas array de S08. En un torneo Abierto sin inscripciones, estas dos últimas listas están vacías. Con rondas, incluye cada batalla S10 para dibujar todo el cuadro. No incluye campeón como propiedad.

Tabla 22. S07 — FinalizacionResponse: JSON y contrato de campos.


```
{
"torneoId": 4,
"estado": "Finalizado",
"campeon": {"id": 7, "nickname":
"anaperez", "elo": 1232}
}
```


torneoId int32; estado string; campeon objeto no null {id:int32,nickname:string,elo:int32}.

Tabla 23. S08 — RondaResponse: JSON y contrato de campos.


```
{
"numero": 2,
"batallas": [
{
"id": 10,
"torneoId": 4,
"ronda": 2,
"jugador1": {"id": 7, "nickname":
"anaperez", "elo": 1200},
"jugador2": {"id": 9, "nickname":
"sixseven", "elo": 1200},
"estado": "Pendiente",
"jugadorTurnoId": 7,
"ganadorId": null
}
]
}
```


numero int32; batallas array de S10 no null. Consulta ordena por número y batalla por id. Generación empareja ELO DESC, desempate id ASC: primero contra último. ELO inicial mayor empieza; empate se sortea.

Tabla 24. S09 — InscripcionResponse: JSON y contrato de campos.

```
{
"id": 12,
"jugadorId": 7,
"nickname": "anaperez",
"elo": 1200,
"torneoId": 4,
"torneo": "Copa Aura",
"estado": "Aceptada",
"fecha": "2026-10-07T12:00:00"
}
```

id/jugadorId/elo/torneoId int32; nickname/torneo/estado strings; fecha DateTime en ISO 8601 (hora local, sin offset garantizado).
elo es actual.

Tabla 25. S10 — BatallaResponse: JSON y contrato de campos.

```
{
"id": 10,
"torneoId": 4,
"ronda": 1,
"jugador1": {"id": 7, "nickname":
"anaperez", "elo": 1200},
"jugador2": {"id": 9, "nickname":
"sixseven", "elo": 1200},
"estado": "Pendiente",
"jugadorTurnoId": 7,
"ganadorId": null
}
```

id/torneoId/ronda int32; jugador1/jugador2 objetos {id,nickname,elo}, ELO actual; estado string; jugadorTurnoId y ganadorId int32 nullable. Turno null si no Pendiente; ganador null si no Finalizada.

Tabla 26. S11 — BatallaDetalleResponse: JSON y contrato de campos.

```
{
"id": 10,
"torneoId": 4,
"ronda": 1,
"jugador1": {"id": 7, "nickname":
"anaperez", "elo": 1216},
"jugador2": {"id": 9, "nickname":
"sixseven", "elo": 1184},
"estado": "Finalizada",
"jugadorTurnoId": null,
"ganadorId": 7,
"comentario": "Victoria por
abandono.",
"cambioEloJugador1": 16,
"cambioEloJugador2": -16,
"lanzamientos": [
{
"turno": 1,
"jugadorId": 7,
"nickname": "anaperez",
"skillId": 3,
"skill": "Six-Seven",
"fecha": "2026-10-07T15:00:00+00:00"
}
]
}
```

Todas las propiedades S10 más comentario string nullable, cambioEloJugador1/cambioEloJugador2 int32 nullable y lanzamientos array S12 ordenado por turno. Antes de fallar comentario/cambios/ganador son null. Los ELO son actuales y los cambios guardados corresponden a ese fallo. Ejemplo: victoria con ELO previo 1200/1200, nuevo 1216/1184.

Tabla 27. S12 — LanzamientoResponse: JSON y contrato de campos.

```
{
"turno": 1,
"jugadorId": 7,
"nickname": "anaperez",
"skillId": 3,
"skill": "Six-Seven",
"fecha": "2026-10-07T15:00:00+00:00"
}
```

turno/jugadorId/skillId int32; nickname/skill strings; fecha DateTimeOffset ISO 8601 con offset. La API registra UTC; turno 1..8.
No expone id propio de lanzamiento.

Tabla 28. S13 — LanzamientoResultadoResponse: JSON y contrato de campos.

<pre>{ "turno": 1, "estado": "Pendiente", "jugadorTurnoId": 9 }</pre>	turno int32 1..8; estado string; jugadorTurnoId int32 nullable. Tras turno 8: estado=EnArbitraje y jugadorTurnoId=null.

Tabla 29. S14 — JugadorRankingResponse: JSON y contrato de campos.

<pre>{ "posicion": 1, "nickname": "anaperez", "elo": 1216, "batallasGanadas": 1, "batallasPerdidas": 0 }</pre>	posicion/elo/batallasGanadas/batallasPerdidas int32; nickname string. Posición inicia en 1; solo jugadores sin baja. Orden ELO DESC/id ASC; conteos globales de batallas Finalizadas.

En S06, un jugadorAceptado real tiene la forma {"jugadorId":7,"nickname":"anaperez","elo":1200}; el nombre de la clave es jugadorId. En S07 y en los jugadores de S10 se usa id. S08 ejemplifica una ronda 2 con dos supervivientes de un cupo inicial 4: una batalla; la primera ronda de ese cupo tiene dos batallas. Son representaciones de estados diferentes y no una respuesta parcialmente truncada.

Antes del fallo, S11 incluye comentario=null, cambioEloJugador1=null y cambioEloJugador2=null; ganadorId también es null. Los objetos de jugadores contienen ELO actual, mientras las variaciones guardadas pertenecen al fallo histórico. El campeón se expone en S07; no se inventa una propiedad campeon en S05/S06.

6. Formato de errores y conflictos

6.1 ErrorResponse y códigos de validación

Tabla 30. ErrorResponse — ErrorResponse: JSON y contrato de campos.

<pre>{ "userMessage": "No existe un torneo con ese id.", "errorCode": "TOURNAMENT_NOT_FOUND", "moreInfo": null, "detalles": null }</pre>	userMessage y errorCode: string. moreInfo: string nullable, actualmente null en todos los productores de error. detalles: array de strings nullable; ModelState incluye los mensajes de validación, MISSING_MANDATORY_SKILLS lista códigos de skills y PLAYERS_MISSING_MANDATORY_SKILL lista nicknames. Si no hay detalles, se devuelve null.

VALIDATION_ERROR es 400 producido por la validación MVC: cuerpo/propiedades requeridas, JSON mal formado y tipos de query no convertibles. INVALID_DATA es 400 de reglas de dominio y conserva el mensaje de la regla. Los errores de autenticación retornan INVALID_TOKEN (401) y los de rol FORBIDDEN (403). En login se usa INVALID_CREDENTIALS. No se devuelve contraseña, token ajeno, stack trace ni información del proveedor SQL.

La Tabla 31 transcribe los mapeos vigentes de excepciones; la Tabla 4 delimita cuáles corresponden a cada operación. Un código mapeado no crea un endpoint ni prueba que todas las operaciones puedan producirlo. Los textos son los userMessage del código, excepto INVALID_DATA, cuyo texto depende de la regla.

Tabla 31. Catálogo de códigos y mensajes de error del estado integrado.

errorCode	HTTP	userMessage / condición
INVALID_DATA	400	Mensaje de la regla de datos inválidos.
INVALID_CREDENTIALS	401	Email o contraseña incorrectos.
USER_NOT_FOUND	404	No existe un usuario con ese id.
SESSION_NOT_FOUND	404	No existe una sesión con ese token.
NOT_SESSION_OWNER	403	La sesión no es tuya.
SKILL_NOT_FOUND	404	No existe una skill con ese id.
DECLARED_SKILL_NOT_FOUND	404	La skill no está en tu repertorio.
TOURNAMENT_NOT_FOUND	404	No existe un torneo con ese id.
EMAIL_ALREADY_EXISTS	409	Ya existe un usuario con ese email.
NICKNAME_ALREADY_EXISTS	409	Ya existe un usuario con ese nickname.
SKILL_CODE_ALREADY_EXISTS	409	Ya existe una skill con ese código.
SKILL_INACTIVE	409	No es posible utilizar la skill porque está inactiva.
SKILL_ALREADY_DECLARED	409	La skill ya está en tu repertorio.
TOURNAMENT_CODE_ALREADY_EXISTS	409	Ya existe un torneo con ese código.
TOURNAMENT_NOT_MODIFIABLE	409	Solo se puede modificar un torneo Abierto.
TOURNAMENT_NOT_CANCELLABLE	409	Solo se puede cancelar un torneo Abierto o En curso.
TOURNAMENT_CAPACITY_BELOW_REGISTRATIONS	409	El cupo no puede ser menor que las inscripciones aceptadas del torneo.
SELF_OPERATION_NOT_ALLOWED	409	Un usuario no puede modificarse ni eliminarse a sí mismo.
NOT_TOURNAMENT_ORGANIZER	403	No sos el organizador asignado a este torneo.
MANDATORY_SKILL_ALREADY_ADDED	409	La skill ya es obligatoria en este torneo.
MANDATORY_SKILLS_LIMIT_REACHED	409	Un torneo puede tener como máximo 2 skills obligatorias.
MANDATORY_SKILL_NOT_FOUND	404	La skill no es obligatoria en este torneo.
MISSING_MANDATORY_SKILLS	409	No tenés declaradas todas las skills obligatorias del torneo.

errorCode	HTTP	userMessage / condición
PLAYERS_MISSING_MANDATORY_SKILL	409	Hay jugadores inscriptos que no tienen declarada la skill.
REGISTRATION_NOT_FOUND	404	No existe una inscripción con ese id.
NOT_REGISTRATION_OWNER	403	La inscripción no es tuya.
TOURNAMENT_NOT_OPEN	409	El torneo no está abierto.
ALREADY_REGISTERED	409	Ya tenés una inscripción aceptada en este torneo.
TOURNAMENT_FULL	409	El torneo ya alcanzó su cupo de inscripciones aceptadas.
REGISTRATION_NOT_ACTIVE	409	La inscripción ya no está aceptada.
MANDATORY_SKILL_IN_USE	409	No podés quitar la skill: es obligatoria en un torneo en el que estás inscripto.
BATTLE_NOT_FOUND	404	No existe una batalla con ese id.
BATTLE_NOT_CANCELLABLE	409	Solo se puede cancelar una batalla Pendiente o En arbitraje.
BATTLE_NOT_PENDING	409	Solo se puede lanzar una skill en una batalla Pendiente.
PLAYER_NOT_IN_BATTLE	409	El jugador no participa de esta batalla.
NOT_PLAYER_TURN	409	No es tu turno para lanzar una skill.
SKILL_NOT_IN_REPERTOIRE	409	La skill no está en tu repertorio.
SKILL_ALREADY_THROWN	409	Ya lanzaste esta skill en la batalla.
BATTLE_CHANGED	409	La batalla cambió mientras se procesaba el pedido. Volvé a consultarla.
MANDATORY_SKILL_PENDING	409	Debés lanzar una skill obligatoria pendiente para completar tus cuatro turnos.
USER_ORGANIZES_ACTIVE_TOURNAMENT	409	El usuario organiza un torneo que no está Finalizado ni Cancelado.
USER_HAS_UNRESOLVED_BATTLES	409	El usuario tiene batallas Pendientes o En arbitraje.
USER_HAS_ACTIVE_REGISTRATION	409	El usuario está inscripto en un torneo que no está Finalizado ni Cancelado.
BATTLE_NOT_JUDGEABLE	409	Solo se puede fallar una batalla Pendiente o En arbitraje.
PLAYER_NOT_IN_TOURNAMENT	403	No tenés una inscripción en este torneo.
TOURNAMENT_CHANGED	409	El torneo cambió mientras se procesaba el pedido. Volvé a consultarlo.
TOURNAMENT_NOT_FULL	409	El torneo todavía no completó su cupo de inscripciones aceptadas.
ROUND_NOT_FOUND	404	El torneo no tiene una ronda con ese número.
ROUND_NOT_RESOLVED	409	La ronda anterior todavía tiene batallas sin resolver.
TOURNAMENT_READY_TO_FINISH	409	Queda un solo jugador sin eliminar: corresponde finalizar el torneo.
TOURNAMENT_NOT_IN_PROGRESS	409	El torneo no está en curso.
TOURNAMENT_HAS_UNRESOLVED_BATTLES	409	El torneo todavía tiene batallas sin resolver.
TOURNAMENT_WITHOUT_CHAMPION	409	Para finalizar el torneo tiene que quedar un solo jugador sin eliminar.
VALIDATION_ERROR	400	Pedido inválido; detalles contiene errores de ModelState.
INVALID_TOKEN	401	Credencial Bearer ausente o inválida; sesión revocada o usuario eliminado.
FORBIDDEN	403	Rol autenticado no habilitado para la operación.
NOT_FOUND	404	No existe el recurso solicitado (routing).
METHOD_NOT_ALLOWED	405	El recurso no admite esa operación; Allow indica métodos publicados.
UNSUPPORTED_MEDIA_TYPE	415	El cuerpo tiene que enviarse en formato JSON.
INTERNAL_ERROR	500	Ocurrió un error inesperado.
REQUEST_NOT_PROCESSED	Variable	Fallback de status sin cuerpo no previsto; no es una respuesta funcional adicional publicada.

6.2 Concurrencia y reconsulta

409 TOURNAMENT_CHANGED puede aparecer al modificar/cancelar un torneo, cambiar obligatorias, inscribir/cancelar inscripciones, generar rondas o finalizar. 409 BATTLE_CHANGED puede aparecer al lanzar, fallar o cancelar el torneo con batallas. Son conflictos detectados durante el procesamiento, aunque el estado consultado previamente permitiera la acción. La versión interna no se recibe como parámetro ni se expone por DTO; no existe contrato If-Match.

Al recibir esos códigos, consultar de nuevo el recurso y decidir si todavía corresponde reenviar. Fallo, ELO, eliminación y versión del torneo se guardan juntos. La generación de una ronda posterior consulta primero la resolución de la anterior y rechaza una lista temporal impar con TOURNAMENT_CHANGED. Dos fallos de batallas distintas pueden competir por la versión del mismo torneo: uno confirma y el otro se rechaza sin efectos parciales. No se declara resuelta toda carrera posible por documentar estos conflictos.

Reglas del resultado: K=32, probabilidad ELO con escala 400, redondeo al entero y mínimo 100 por jugador. Tras el fallo, el perdedor pasa a Eliminada en su inscripción. Cuatro turnos por jugador, sin repetir skill; si obligatorias pendientes coincide con turnos restantes, el lanzamiento debe ser obligatorio. No se bloquea una skill declarada por estar inactiva posteriormente. Finalizar exige EnCurso, ninguna batalla sin resolver y exactamente una inscripción Aceptada.

7. Uso de IA

El equipo usó IA (Claude Code y Codex) como herramienta para acortar los tiempos de trabajo y cumplir con los plazos de entrega: ayudó a revisar la especificación contra el código y a completar los ejemplos y el catálogo de errores. Las decisiones de diseño de la API son del equipo; revisamos todo lo generado contra el código y la bitácora, y podemos explicar cada decisión.

8. Bibliografía

[1] Universidad ORT Uruguay, Facultad de Ingeniería, "Obligatorio 1 – Diseño de Aplicaciones 2, 2026.2," letra del obligatorio, Montevideo, 2026.

[2] J. Ferreiro, S. Luciardi y A. Bustamante, "M6CD-ID-274754-328344-302420," repositorio en GitHub, 2026. [Online]. Available: https://github.com/IngSoft-DA2/M6CD-ID-274754-328344-302420 [Accedido: 8 de octubre de 2026].

[3] Universidad ORT Uruguay, Diseño de Aplicaciones 2, "Módulo M1: REST," material del curso, Montevideo, 2026.

[4] J. Ferreiro, S. Luciardi y A. Bustamante, "Bitácora de decisiones de FarmeORT," en [2], 2026. [Online]. Available: https://github.com/IngSoft-DA2/M6CD-ID-274754-328344-302420/blob/develop/docs/bitacora-decisiones.md [Accedido: 8 de octubre de 2026].

[5] J. Ferreiro, S. Luciardi y A. Bustamante, "Colección de Postman de FarmeORT," en [2], 2026. [Online]. Available: https://github.com/IngSoft-DA2/M6CD-ID-274754-328344-302420/blob/develop/Datos/FarmeORT.postman_collection.json [Accedido: 8 de octubre de 2026].